EXP-9

Student Name: Srijan Tripathi **UID:** 24BAI70081

Branch: AIT-CSE (AIML) **Section/Group:** 24AIT_KRG-1A

Semester: 5 **Subject Code:** 24CSP-310

Subject Name : Advanced Database Management System

1.

- An organization stores employee details in the employee_info table. Create a PostgreSQL stored procedure that accepts an employee's **gender as an input parameter** and calculates the **total number of employees belonging to that gender**. The calculated count should be returned through an **OUT parameter** and should also be displayed as a message using RAISE NOTICE.
- For example, if Male is passed to the procedure, it should count all employees whose gender is Male.

employee_info

Column Name	Data Type	Description
id	SERIAL	Unique employee ID
name	VARCHAR(50)	Employee name
gender	VARCHAR(10)	Gender of employee
salary	NUMERIC(10,2)	Employee salary
city	VARCHAR(50)	Employee city

Code:

```sql
CREATE TABLE employee_info (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    gender VARCHAR(10) NOT NULL,
    salary NUMERIC(10,2) NOT NULL,
    city VARCHAR(50) NOT NULL
);
INSERT INTO employee_info (name, gender, salary, city)
VALUES
('Alok', 'Male', 50000.00, 'Delhi'),
('Priya', 'Male', 60000.00, 'Mumbai'),
('Rajesh', 'Female', 45000.00, 'Bangalore'),
('Sneha', 'Male', 55000.00, 'Chennai'),
('Anil', 'Male', 52000.00, 'Hyderabad'),
('Sunita', 'Female', 48000.00, 'Kolkata'),
('Vijay', 'Male', 47000.00, 'Pune'),
('Ritu', 'Male', 62000.00, 'Ahmedabad'),
('Amit', 'Female', 51000.00, 'Jaipur');
```

```sql
CREATE OR REPLACE PROCEDURE SP_COUNT_GENDER(IN P_GENDER VARCHAR, OUT P_TOTAL_COUNT INT)
LANGUAGE PLPGSQL
AS
$$
BEGIN
    SELECT COUNT(ID)
    INTO P_TOTAL_COUNT
    FROM EMPLOYEE_INFO
    WHERE GENDER = P_GENDER;

    RAISE NOTICE 'THE COUNT OF EMPLOYEE HAVING GENDER AS %  ARE %', P_GENDER, P_TOTAL_COUNT;
END;
$$;

--CALLING THE STORED PROCEUDRE
CALL SP_COUNT_GENDER('Female', NULL);
```

Output:

	p_total_count integer
1	3

2.

An organization stores employee details in the employee_info table. Create a PostgreSQL stored procedure that accepts an employee's **gender as an input parameter** and calculates the **total number of employees belonging to that gender**. The calculated count should be returned through an **OUT parameter** and should also be displayed as a message using RAISE NOTICE.

For example, if Male is passed to the procedure, it should count all employees whose gender is Male.

employee_info

Column Name	Data Type	Description
id	SERIAL	Unique employee ID
name	VARCHAR(50)	Employee name
gender	VARCHAR(10)	Gender of employee
salary	NUMERIC(10,2)	Employee salary
city	VARCHAR(50)	Employee city

Code:

```
CREATE TABLE products (
    product_code VARCHAR(10) PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price NUMERIC(10,2) NOT NULL,
    quantity_remaining INT NOT NULL,
    quantity_sold INT DEFAULT 0
);

CREATE TABLE sales (
    order_id SERIAL PRIMARY KEY,
    order_date DATE NOT NULL,
    product_code VARCHAR(10) NOT NULL,
    quantity_ordered INT NOT NULL,
    sale_price NUMERIC(10,2) NOT NULL,
    FOREIGN KEY (product_code) REFERENCES products(product_code)
);
```






```sql
INSERT INTO products (product_code, product_name, price, quantity_remaining, quantity_sold)
VALUES
('P001', 'iPHONE 13 PRO MAX', 109999.00, 10, 0),
('P002', 'Samsung Galaxy S23 Ultra', 99999.00, 8, 0),
('P003', 'iPAD AIR', 55999.00, 5, 0),
('P004', 'MacBook Pro 14"', 189999.00, 3, 0),
('P005', 'Sony WH-1000XM5 Headphones', 29999.00, 15, 0);

INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)
VALUES
('2025-09-15', 'P001', 1, 109999.00),
('2025-09-16', 'P002', 2, 199998.00),
('2025-09-17', 'P003', 1, 55999.00),
('2025-09-18', 'P005', 2, 59998.00),
('2025-09-19', 'P001', 1, 109999.00);
```

```sql
CREATE OR REPLACE PROCEDURE pr_buy_products(
    IN p_product_name VARCHAR,
    IN p_quantity INT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_product_code VARCHAR(20);
    v_price FLOAT;
    v_count INT;
BEGIN
    -- Step 1: Check if product exists and has enough quantity
    SELECT COUNT(*)
    INTO v_count
    FROM products
    WHERE product_name = p_product_name
    AND quantity_remaining >= p_quantity;
    -- Step 2: If sufficient stock
    IF v_count > 0 THEN
        -- Fetch product code and price
        SELECT product_code, price
        INTO v_product_code, v_price
        FROM products
        WHERE product_name = p_product_name;
        -- Insert a new record into the sales table
        INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)
        VALUES (CURRENT_DATE, v_product_code, p_quantity, (v_price * p_quantity));
        -- Update stock details
        UPDATE products
        SET quantity_remaining = quantity_remaining - p_quantity,
            quantity_sold = quantity_sold + p_quantity WHERE product_code = v_product_code;
        -- Confirmation message
        RAISE NOTICE 'PRODUCT SOLD..! Order placed successfully for % unit(s) of %.', p_quantity, p_product_name;
    ELSE
        -- Step 3: If stock is insufficient
        RAISE NOTICE 'INSUFFICIENT QUANTITY..! Order cannot be processed for % unit(s) of %.', p_quantity, p_product_name;
    END IF;
END;
$$;
CALL pr_buy_products ('MacBook Pro 14"', 1);
```

Output:

```
NOTICE:  PRODUCT SOLD..! Order placed successfully for 1 unit(s) of MacBook Pro 14".
CALL

Query returned successfully in 84 msec.
```

LEARNING OUTCOMES:

- Understand the purpose and structure of stored procedures in PostgreSQL.
- Create procedures using IN, OUT, and INOUT parameters.
- Implement procedural logic using PL/pgSQL, including variables, conditions, and SQL queries.
- Execute stored procedures using the CALL statement and handle output parameters.
- Use statements such as SELECT INTO and RAISE NOTICE to process and display results.

CONCLUSION:

Stored procedures provide a structured way to store and execute database operations on the server side. By using parameters, SQL statements, and PL/pgSQL control structures, complex database tasks can be organized into reusable procedures. This improves code reusability, reduces repetitive queries, and provides an efficient way to perform operations directly within the database.